

Condensation Tracker

Ziyu Ma

Task1: Implementation

1.Color histogram

```
import numpy as np

def color_histogram(xmin, ymin, xmax, ymax, frame, hist_bin):

    bb_frame = frame[ymin:ymax, xmin:xmax, :]

    color_hist = np.zeros((3, hist_bin))

    color_hist[0] = np.histogram(bb_frame[:, :, 0], bins=hist_bin)[0]
    color_hist[1] = np.histogram(bb_frame[:, :, 1], bins=hist_bin)[0]
    color_hist[2] = np.histogram(bb_frame[:, :, 2], bins=hist_bin)[0]

    color_hist = color_hist/np.sum(color_hist)

    return color_hist
```

The function bins each color channel (R,G,B) into *hist_bin* bins for the bounding box defined by (*xmin, xmax*) (*ymin, ymax*) of the current frame. In the end, the histogram is normalized.

2&3. Derive matrix A & Propagation

```
import numpy as np

def propagate(particles, frame_height, frame_width, params):

    #position noise
    w = np.random.normal(loc=0, scale=params["sigma_position"], size=(2,
particles.shape[0]))

    #A matrices
    if params["model"] == 0:
        A = np.array([[1, 0], [0, 1]])

    if params["model"] == 1:
        A = np.array([[1, 0, 1, 0], [0, 1, 0, 1], [0, 0, 1, 0], [0, 0, 0, 1]])
        #velocity noise
        w_vel = np.random.normal(loc=0, scale=params["sigma_velocity"], size=(2,
particles.shape[0]))
        w = np.vstack((w,w_vel))

    #propagate
    propa_particles = (np.matmul(A, particles.T) + w).T

    #make sure the center inside the frame
    propa_particles[:,0] = np.clip(propa_particles[:,0], 0, frame_width-1)
    propa_particles[:,1] = np.clip(propa_particles[:,1], 0, frame_height-1)

    return propa_particles
```

No motion at all: A is simply an identity matrix. Identity multiplied with the particle position remains the particle position.

Constant velocity motion model: A is $\{\{1,0,1,0\}, \{0,1,0,1\}, \{0,0,1,0\}, \{0,0,0,1\}\}$. Multiplied with $s = \{x, y, v_x, v_y\}$ gives $\{x+v_x, y+v_y, v_x, v_y\}$. It means move the position in the direction of the velocity.

Propagate function: The formula $s_{\text{new}} = A*s_{\text{old}} + \text{noise}$ is used. If $\text{params.model} = 0$, no

motion model is used. The noise is simply the normal distribution with the std. dev. `params.sigma_position`. If the constant velocity model is used, then besides the position noise, there is also a normal distributed velocity noise with std. dev. `params.sigma_velocity`. In the end, clip to make sure that the particles lie inside the frame.

4. Observation

```
import numpy as np
from color_histogram import color_histogram
from chi2_cost import chi2_cost

def observe(particles, frame, bbox_height, bbox_width, hist_bin, hist,
            sigma_observe):

    particles_w = np.zeros((particles.shape[0],1))

    frame_height = frame.shape[0]
    frame_width = frame.shape[1]

    for i in range(particles.shape[0]):
        xmin = min(max(0, round(particles[i,0] - 0.5*bbox_width)), frame_width-1)
        xmax = min(max(0, round(particles[i,0] + 0.5*bbox_width)), frame_width-1)
        ymin = min(max(0, round(particles[i,1] - 0.5*bbox_height)), frame_height-1)
        ymax = min(max(0, round(particles[i,1] + 0.5*bbox_height)), frame_height-1)
        hist_i = color_histogram(xmin, ymin, xmax, ymax, frame, hist_bin)
        chi2_i = chi2_cost(hist_i, hist)
        particles_w[i,:] = 1/(np.sqrt(2*np.pi)*sigma_observe) * np.exp(-
            chi2_i**2/(2*sigma_observe**2))

    particles_w = particles_w/np.sum(particles_w)

    return particles_w
```

In this function, `xmin`, `xmax`, `ymin` and `ymax` are calculated for each particle and are made sure to be inside the frame. Then for each particle, the color histogram is calculated and compared with the target histogram to get the weight using chi2 distance. In the end, the weights are normalized.

5. Estimation

```
import numpy as np

def estimate(particles, particles_w):

    mean_state = np.sum(particles*particles_w, axis=0)/np.sum(particles_w)
    return mean_state
```

This function simply multiplies `particles` and `particles_w` elementwise and returns a normalized sum.

6. Resampling

```
import numpy as np

def resample(particles, particles_w):

    sample_indices = np.random.choice(particles.shape[0], size=particles.shape[0],
        replace=True, p=particles_w.flatten())

    sample_particles = particles[sample_indices]
    sample_particles_w = particles_w[sample_indices] /
        np.sum(particles_w[sample_indices])
    return sample_particles, sample_particles_w
```

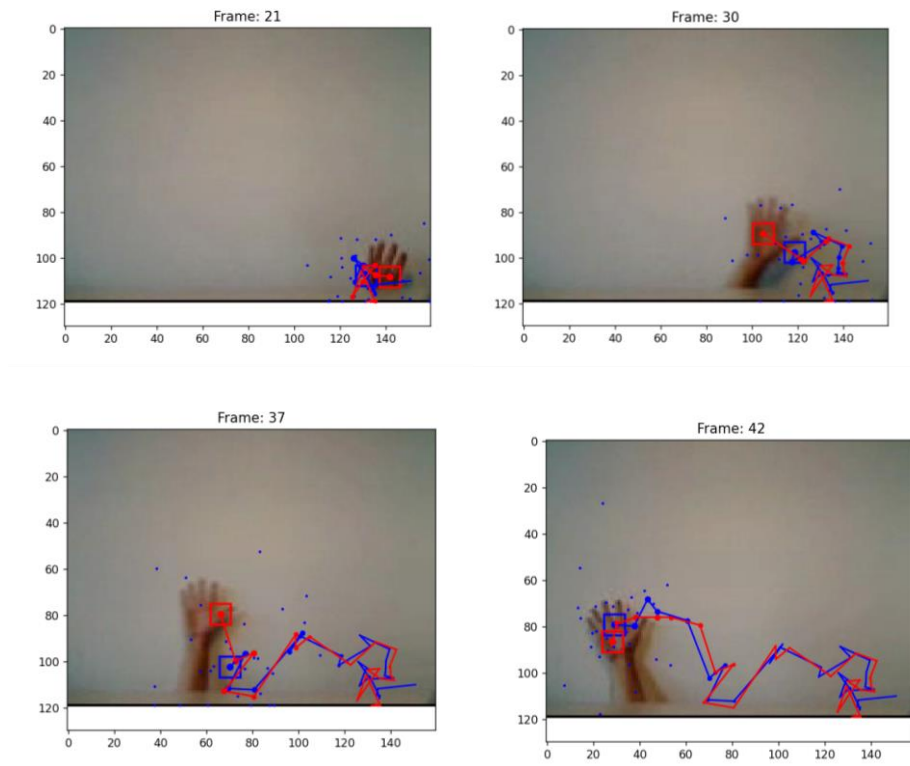
`np.random.choice` is used to sample according to the `particles_w` weights.

Task2: Experiments

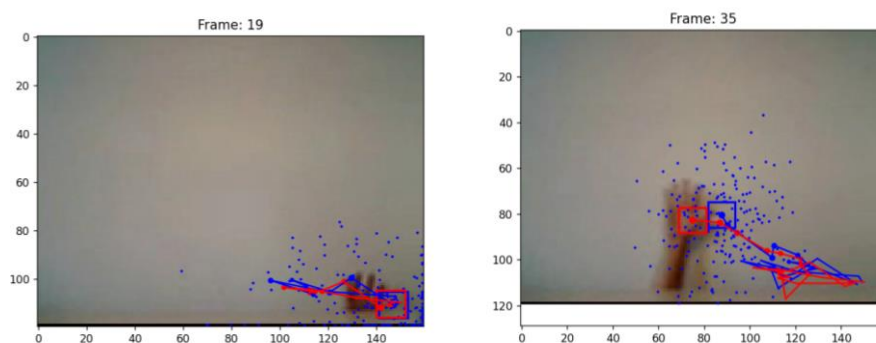
Video 1: As an example, results of Nr.1 and Nr.2 are shown. We could see that more particles lead to better tracking since the trace in the middle part of Nr.2 doesn't go downwards (as in the original video), in contrast to Nr.1.

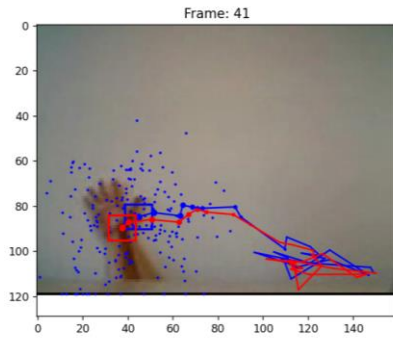
Nr.	hist_bin	alpha	sigma_observe	model	num_particles	sigma_position	sigma_velocity	initial_velocity
1	16	0	0.1	0	10	15	1	(1,10)
2	16	0	0.1	0	200	15	1	(1,10)

Nr.1:



Nr.2:



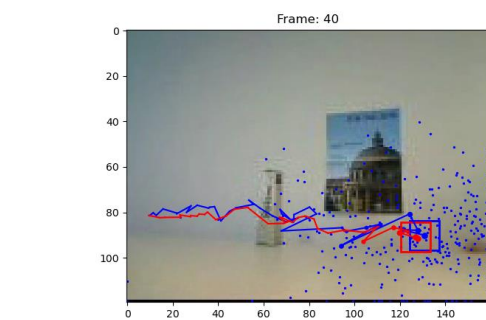


Video2:

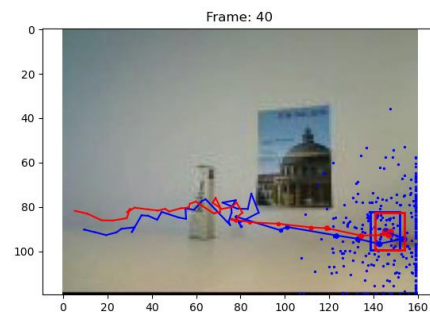
Q: What is the effect of using a constant velocity motion model?

A: From the Nr.3, Nr.4 and Nr.5 we can see that accurate estimate of the initial velocity will increase the accuracy (less lag after occlusion, see Nr.3 and Nr.4), because it uses the fact that the object keep move in the old direction. The accuracy of the initial velocity is important, otherwise worse results (severe lag after occlusion, see Nr.4 and Nr.5).

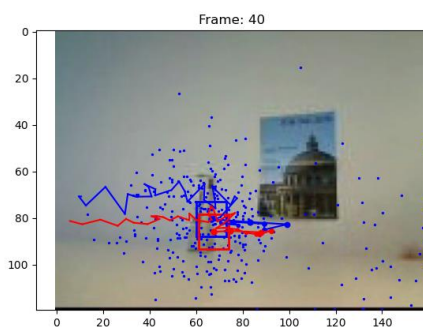
Nr.	hist_bin	alpha	sigma_observe	model	num_particles	sigma_position	sigma_velocity	initial_velocity
3	16	0	0.1	0	300	15	1	(1,10)
4	16	0	0.1	1	300	15	1	(1,10)
5	16	0	0.1	1	300	15	1	(1,-10)



Nr.3



Nr.4



Nr.5

Q: What is the effect of assuming decreased/increased system noise?

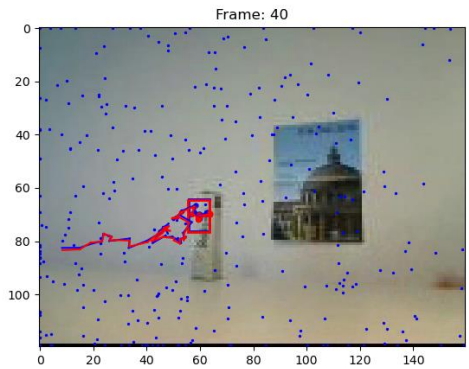
A: High system noise spreads the particles apart. It can help to track better when the target object is partly occluded, since some particles still land on it. However, having it too high leads

to bad performance since the particles focus less on the target object but the environment, and are thus distracted.

Q: What is the effect of assuming decreased/increased measurement noise?

A: With high measurement noise the weights are more spread. Before, the resampling part chooses particles with small difference to the target feature more often, leads the bounding box to follow the target object. But now particles differ a lot from the target feature are also given importance, leads to loss of track of the object.

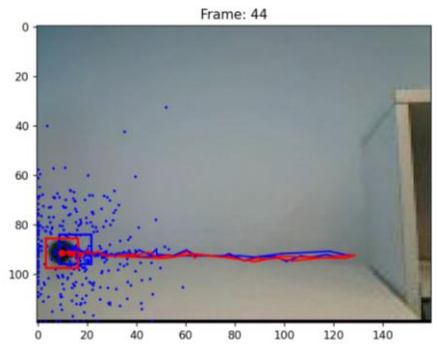
Nr,	hist_bin	alpha	sigma_observe	model	num_particles	sigma_position	sigma_velocity	initial_velocity
6	16	0	0.9	0	300	15	1	(1,10)



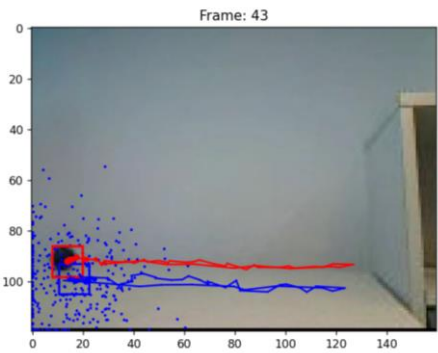
Nr.6

Video3:

Nr,	hist_bin	alpha	sigma_observe	model	num_particles	sigma_position	sigma_velocity	initial_velocity
7	16	0	0.1	0	300	15	1	(1,10)
8	16	0	0.1	1	300	15	1	(1,10)

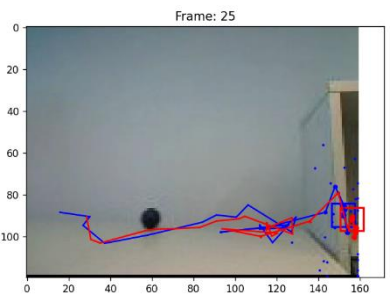


Nr.7



Nr.8

Nr,	hist_bin	alpha	sigma_observe	model	num_particles	sigma_position	sigma_velocity	initial_velocity
9	16	0	0.1	0	30	15	1	(1,10)



Nr.9

show this because we have to show one unsuccessful case.

With the best parameters from video2 the ball can still be tracked, but the performance is not improved. The constant velocity model is not suitable for video3, since the ball velocity reverts direction once it hits the shelf.

Q: What is the effect of using a constant velocity motion model?

A: stated above.

Q: What is the effect of assuming decreased/increased system noise?

A: As stated above, with increased system noise the particles are spread apart. Since we have no occlusion here, this is not needed. We better have small value to concentrate.

Q: What is the effect of assuming decreased/increased measurement noise?

A: similar as for video2. Here we have no occlusion, thus no need to consider the case that the particles' histograms have to differ from the target's while occluding. Using a small value let us focus on the target histogram of the ball.

Questions:

Q: What is the effect of using more or fewer particles?

A: More particles lead to better results but also more computational cost.

Q: What is the effect of using more or fewer bins in the histogram color model?

A: Increasing the bins in a certain amount increases performance, but too much would make the computation not robust to noise.

Q: What is the advantage/disadvantage of allowing appearance model updating?

A: When the color histogram of the object changes while moving, using a bigger alpha helps correcting (e.g. when the lighting changes or moving in front of a background). However, it might lose the object in the occlusion case, since the target feature is changed to the object in front of the target object gradually.